

SecureStream-86 – Deterministic Security Token Generator

Team: Binary Heart

Studenti: Şendroi Ana Teodora
Zăgrean Sonia Andreea
Grupa 323, Matematică-Informatică

GitHub: <https://github.com/soniazagrean/AsmProject>

1. Descriere generală

Proiectul **SecureStream-86** este un program scris în limbaj de asamblare pentru arhitectura 8086 (16 biți), având ca scop exersarea operațiilor aritmetice și logice pe biți, manipularea șirurilor de date și utilizarea întreruperilor DOS pentru intrare/ieșire.

Programul citește din fișier un șir de 8–16 octeți introduși în format hexazecimal, îi procesează conform unor reguli bine definite și generează un token de securitate determinist (cuvântul C), urmat de sortarea și shiftarea datelor.

Proiectul este structurat modular, folosind subrutine, pentru claritate, reutilizare și ușurință în testare.

2. Structura generală a programului

Programul este organizat sub forma unui pipeline de date, cu următoarele etape:

- Citirea și conversia datelor de intrare
- Calculul cuvântului de securitate C
- Manipularea șirului de octeți (sortare și analiză)
- Shuffle prin rotații circulare
- Afișarea rezultatelor

Fiecare etapă este implementată într-o subrutină dedicată.

3. Citirea datelor din fișier

Datele de intrare nu mai sunt preluate de la tastatură, ci sunt citite dintr-un fișier extern ‘input.txt’. Procesul urmează etapele specifice lucrului cu fișiere în mediul DOS:

- Deschiderea fișierului: Se utilizează întreruperea INT 21h, funcția AH=3Dh pentru a obține un handle de fișier.
- Citirea conținutului: Se folosește funcția AH=3Fh, care citește un număr specificat de octeți (corespunzător formatului hexazecimal) și îi stochează într-un buffer temporar.
- Închiderea fișierului: După preluarea datelor, fișierul este închis folosind funcția AH=3Eh pentru a elibera resursele sistemului.

Conversia și Stocarea

- Conversia ASCII → Binar: Caracterele citite din fișier sunt convertite în valori numerice:
 - '0'-'9' devin valori 0–9
 - 'A'-'F' sau 'a'-'f' devin valori 10–15
- Formarea Octeților: Fiecare pereche de două caractere hexazecimale este combinată pentru a forma un singur octet binar (3Fh), care este apoi stocat în șirul principal din segmentul de date.
- Validarea: Programul verifică dacă au fost extrași între 8 și 16 octeți valizi din fișier pentru a putea continua procesarea.

4. Calculul cuvântului C (16 biți)

Cuvântul C este construit conform cerinței folosind operații bitwise și aritmetice:

- 4.1 Biții 0–3 - se calculează XOR între:
 - primii 4 biți ai primului octet
 - ultimii 4 biți ai ultimului octet
- 4.2 Biții 4–7 - se realizează operația OR între biții 2–5 ai fiecărui octet din șir.
 - rezultatul este limitat la 4 biți.
- 4.3 Biții 8–15 - se calculează suma tuturor octeților modulo 256.
 - partea superioară a cuvântului C este formată din această sumă.

Rezultatul final este un cuvânt de 16 biți care reprezintă token-ul de securitate determinist.

5. Manipularea șirului de octeți

- 5.1 Sortarea descrescătoare
Șirul de octeți este sortat în ordine descrescătoare folosind algoritmul Bubble Sort. Acest algoritm a fost ales pentru simplitate și claritate în implementare.
- 5.2 Determinarea octetului cu valoare mare
Se parcurge șirul pentru a determina octetul care conține cel mai mare număr de biți de valoare 1, cu condiția ca acest număr să fie mai mare de 3.

Numărul de biți 1 este calculat prin deplasări succesive și testarea flag-ului Carry.

6. Rotirea valorilor

Pentru fiecare octet din șir se aplică o operație de shuffle:

1. Se calculează valoarea N ca suma primilor doi biți ai octetului
2. Octetul este rotit circular la stânga cu N poziții, folosind instrucțiunea **ROL**

Această etapă modifică datele inițiale într-un mod determinist, dar dificil de intuit fără cunoașterea

regulilor interne.

7. Afișarea rezultatului

Programul afișează:

- Cuvântul C în format hexazecimal
- Șirul de octeți rezultat după sortare și rotiri
- Valorile sunt afișate atât în hexazecimal, cât și în binar

Afișarea se realizează folosind întreruperea DOS INT 21h, AH = 02h / 09h.

8. Dificultăți întâmpinate și soluții

Erori la Deschiderea Fișierului: O dificultate majoră a fost gestionarea situației în care fișierul lipsește sau are calea greșită. Soluția a constat în verificarea **Carry Flag (CF)** după apelul funcției 3Dh și afișarea unui mesaj de eroare corespunzător în cazul în care fișierul nu a putut fi deschis.

Parsarea Buffer-ului: Deoarece datele din fișier pot conține spații sau caractere de tip *new-line*, a fost necesară implementarea unei logici care să sară peste caracterele non-hex pentru a extrage corect doar valorile numerice.

Modularitatea: Utilizarea subrutinelor a fost esențială pentru a nu aglomera segmentul de cod principal, separând logica de citire a fișierului de logica de prelucrare a biților.

9. Concluzie

Proiectul SecureStream-86 demonstrează utilizarea eficientă a limbajului de asamblare 8086 pentru procesarea datelor la nivel de biți, manipularea șirurilor și realizarea unui program interactiv complet funcțional.

Structura modulară și comentariile clare fac codul ușor de înțeles, testat și extins, îndeplinind toate cerințele impuse în temă.